

# КОНСТРУЮВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

Спеціальність 121 — Інженерія програмного забезпечення

## МОДУЛЬ 3

### Пишемо реальний код з практиками індустрії

5 практичних робіт · 10 академічних годин · 3 курс

|                                           |                                             |                                      |                                            |                                          |
|-------------------------------------------|---------------------------------------------|--------------------------------------|--------------------------------------------|------------------------------------------|
| <b>ПР-8</b><br>Патерни: Factory & Builder | <b>ПР-9</b><br>Патерни: Observer & Strategy | <b>ПР-10</b><br>Помилки та логування | <b>ПР-11</b><br>Code review та рефакторинг | <b>ПР-12</b><br>Асинхронне програмування |
|-------------------------------------------|---------------------------------------------|--------------------------------------|--------------------------------------------|------------------------------------------|

3 курс · 2026 навчальний рік

## Вступ до модуля 3

Модулі 1 і 2 навчили вас правильно готуватися до розробки: налаштовувати середовище, описувати вимоги, проектувати архітектуру й базу даних. Тепер настав час написати перший реальний код — і написати його правильно.

«Правильно» означає не просто «без синтаксичних помилок». Це означає код, якому можна довіряти: він передбачувано обробляє помилки, реєструє події, легко читається і побудований на перевірених рішеннях (патернах). Саме такий код пишуть у командах.

### АНАЛОГІЯ Аналогія з реального життя

Уявіть двох будівельників. Перший кладе цеглу інтуїтивно, на свій розсуд. Другий використовує схеми, перевірені матеріали і стандартні вузли з'єднань. Перший може побудувати швидше — але коли будинок потрібно розширити або відремонтувати, виявляється, що ніхто крім автора не розуміє конструкцію. Другий будує трохи довше, але будь-який інший будівельник зможе продовжити роботу. Патерни проектування — це і є стандартні вузли з'єднань у програмуванні.

У цьому модулі ви навчитеся:

- Застосовувати породжуючі та поведінкові патерни GoF у реальному коді
- Будувати надійну систему обробки помилок і структурованого логування
- Проводити code review та виявляти code smells у чужому коді
- Писати асинхронний код, що не блокує виконання програми

### ІНДУСТРІЯ Як це використовують у роботі

За даними JetBrains Developer Ecosystem Survey 2024: патерни GoF знають і застосовують 78% досвідчених розробників. Логування і моніторинг — вимога до 92% production-систем. Асинхронне програмування стало обов'язковою навичкою: `async/await` використовують у 87% Node.js проєктів і 74% Python проєктів.

## 1. Теоретична частина

### 1.1. Що таке патерн проектування

Патерн проектування (Design Pattern) — це перевірений, багаторазово використовуваний спосіб вирішення типової проблеми в об'єктно-орієнтованому проектуванні. Поняття популяризовано книгою «Design Patterns: Elements of Reusable Object-Oriented Software» (1994) — «Банда чотирьох» (GoF: Gang of Four): Гамма, Хелм, Джонсон, Вліссідес.

Важливо розуміти: патерн — не готовий код для копіювання. Це загальна ідея рішення, яку ви адаптуєте під свою задачу. Як рецепт: написано «додайте сіль за смаком», а не «додайте рівно 3,7 грами солі».

#### ТЕОРІЯ Ключова теорія

Класифікація патернів GoF (23 патерни у трьох групах):

- Породжуючі (Creational) — керують створенням об'єктів: Singleton, Factory Method, Abstract Factory, Builder, Prototype
- Структурні (Structural) — організують відносини між об'єктами: Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy
- Поведінкові (Behavioral) — керують взаємодією та відповідальністю: Chain of Responsibility, Command, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method, Visitor

#### ІНДУСТРІЯ Як це використовують у роботі

Патерни GoF — спільна мова розробників. Коли в команді кажуть 'зроби це через Observer' або 'тут потрібен Facade' — всі розуміють про що мова без довгих пояснень. Це як математичні символи: замість 'сума квадратів різниць' — просто  $\sigma^2$ .

## 1.2. Патерн Singleton — один екземпляр на всю програму

Singleton (одинак) гарантує, що клас має лише один екземпляр, і надає глобальну точку доступу до нього. Застосовується, коли потрібен рівно один об'єкт певного типу на всю програму.

### АНАЛОГІЯ Аналогія з реального життя

Уявіть президента країни. Президент може бути лише один одночасно. Якщо хтось запитує 'хто президент?' — завжди отримує посилання на одну й ту саму людину, а не нову копію. Singleton — це і є 'президент' серед об'єктів: завжди один, завжди один і той самий.

Коли застосовувати Singleton: з'єднання з базою даних (не створювати нове з'єднання при кожному запиті), конфігурація програми (один об'єкт налаштувань), системний логер (один логер для всього застосунок), менеджер черги задач.

```
# Python: Singleton через __new__
class DatabaseConnection:
    _instance = None    # зберігає єдиний екземпляр

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance._connected = False
        return cls._instance    # завжди повертаємо той самий

    def connect(self, url: str):
        if not self._connected:
            print(f"Підключення до {url}")
            self._connected = True
        else:
            print("З'єднання вже існує, повертаємо кешоване")

    def query(self, sql: str):
        return f"Результат: {sql}"

# Перевірка: db1 і db2 — один і той самий об'єкт
db1 = DatabaseConnection()
db2 = DatabaseConnection()

db1.connect("postgresql://localhost/mydb")
db2.connect("postgresql://localhost/mydb")    # не створить нове з'єднання

print(db1 is db2)    # True — це один об'єкт!
print(id(db1) == id(db2))    # True
```

## УВАГА Важливо знати

Singleton — один з найбільш критикованих патернів, бо порушує принцип тестування: якщо Singleton зберігає стан, він 'забруднює' тести між собою. Сучасне рішення — використовувати Dependency Injection (DIP з Модуля 2): передавати з'єднання через конструктор, а не отримувати глобально. Але для конфігурації та логера Singleton — досі прийнятний вибір.

### 1.3. Патерн Factory Method — делегуємо рішення про клас нащадкам

Factory Method (фабричний метод) визначає інтерфейс для створення об'єкта, але дозволяє підкласам вирішувати, який клас інстанціювати. Вирішує проблему: як створювати об'єкти, не прив'язуючись до конкретних класів.

## АНАЛОГІЯ Аналогія з реального життя

Уявіть піцерію. Базовий процес однаковий: замісити тісто, додати начинку, запекти. Але яка саме піца — вирішує конкретна піцерія. 'Піца Маргарита' (підклас) знає свою начинку. 'Піца Пепероні' знає свою. Базовий клас `PizzaStore` (фабрика) каже 'зроби піцу', а конкретний підклас вирішує яку саме.

```
# Python: Factory Method для нотифікацій
from abc import ABC, abstractmethod

# Абстрактний продукт
class Notifier(ABC):
    @abstractmethod
    def send(self, recipient: str, message: str) -> bool:
        pass

# Конкретні продукти
class EmailNotifier(Notifier):
    def send(self, recipient: str, message: str) -> bool:
        print(f"EMAIL -> {recipient}: {message}")
        return True

class SMSNotifier(Notifier):
    def send(self, recipient: str, message: str) -> bool:
        print(f"SMS -> {recipient}: {message}")
        return True

class TelegramNotifier(Notifier):
    def send(self, recipient: str, message: str) -> bool:
```

```

        print(f"TG      -> {recipient}: {message}")
        return True

# Фабрика — повертає потрібний нотифікатор
class NotifierFactory:
    _notifiers = {
        "email": EmailNotifier,
        "sms": SMSNotifier,
        "telegram": TelegramNotifier,
    }

    @classmethod
    def create(cls, channel: str) -> Notifier:
        notifier_class = cls._notifiers.get(channel.lower())
        if not notifier_class:
            raise ValueError(f"Невідомий канал: '{channel}'")
        return notifier_class()

# Клієнтський код — не знає про конкретні класи
channel = "telegram"          # може прийти з конфігурації або БД
notifier = NotifierFactory.create(channel)
notifier.send("user@example.com", "Ваша задача виконана!")

# Додати новий канал = додати один клас + один рядок у словник
# Клієнтський код не змінюється! (принцип ОСР з Модуля 2)

```

#### ПРИКЛАД Практичний приклад

Де Factory Method використовується в популярних бібліотеках: SQLAlchemy — `create_engine('postgresql://...')` повертає різні об'єкти залежно від URL. Django ORM — `objects.get()`, `objects.filter()` через Manager (внутрішня фабрика). FastAPI — `Depends()` — фабрика залежностей. `logging.getLogger()` — фабрика логерів.

### 1.4. Патерн Builder — покрокове конструювання складних об'єктів

Builder (будівник) відокремлює конструювання складного об'єкта від його представлення, дозволяючи одному й тому ж процесу конструювання створювати різні представлення. Вирішує проблему конструктора з багатьма параметрами (Telescoping Constructor Antipattern).

## АНАЛОГІЯ Аналогія з реального життя

Уявіть конфігуратор автомобіля на сайті. Ви крок за кроком обираєте: кузов → двигун → колір → опції. Не обов'язково вказувати всі параметри одразу. Можна взяти базову комплектацію і дорозібрати пізніше. Builder — саме цей конфігуратор у коді.

```
# Python: Builder для HTTP-запиту (спрощений приклад)
class HttpRequest:
    """Об'єкт, який будуємо"""
    def __init__(self):
        self.method = "GET"
        self.url = ""
        self.headers = {}
        self.params = {}
        self.body = None
        self.timeout = 30

    def __repr__(self):
        return (f"HttpRequest({self.method} {self.url} "
                f"headers={self.headers} timeout={self.timeout}s)")

class HttpRequestBuilder:
    """Будівник — додаємо методи ланцюжком"""
    def __init__(self, url: str):
        self._request = HttpRequest()
        self._request.url = url

    def method(self, method: str):
        self._request.method = method.upper()
        return self # ← повертаємо self для ланцюжка

    def header(self, key: str, value: str):
        self._request.headers[key] = value
        return self

    def param(self, key: str, value: str):
        self._request.params[key] = value
        return self

    def body(self, data: dict):
        self._request.body = data
        return self

    def timeout(self, seconds: int):
        self._request.timeout = seconds
```

```

        return self

    def build(self) -> HttpRequest:
        if not self._request.url:
            raise ValueError("URL є обов'язковим")
        return self._request

# Використання — читається як природна мова:
request = (
    HttpRequestBuilder("https://api.example.com/tasks")
    .method("POST")
    .header("Authorization", "Bearer my_token")
    .header("Content-Type", "application/json")
    .body({"title": "Нова задача", "priority": "high"})
    .timeout(10)
    .build()
)
print(request)
# HttpRequest(POST https://api.example.com/tasks
#           headers={'Authorization': ..., 'Content-Type': ...}
#           timeout=10s)

```

### ПРИКЛАД Практичний приклад

Builder у реальних бібліотеках: SQLAlchemy Query Builder — `db.query(User).filter(User.active==True).order_by(User.name).limit(10)`. Elasticsearch Python Client — `Q('match', title='python') & Q('range', views={'gte':100})`. Pytest fixtures — `RequestBuilder().with_auth().with_json_body()`. Всі вони використовують ланцюжок методів (method chaining) — signature Builder.

### ПАМ'ЯТКА Коротко і по суті

- Singleton: один екземпляр — для БД з'єднання, конфігурації, логера
- Factory Method: вибрати клас у runtime — коли тип залежить від даних або конфігурації
- Builder: складний об'єкт крок за кроком — коли є багато необов'язкових параметрів
- Ключова ознака Builder: методи повертають self — можна писати ланцюжком



## 2. Практичне завдання ПР-8

### ПРАКТИЧНЕ ЗАВДАННЯ 8.1 Реалізація трьох породжуючих патернів у проєкті

Мета: реалізувати Singleton, Factory Method та Builder у коді вашого наскрізного проєкту. Кожен патерн повинен вирішувати реальну задачу проєкту, а не бути вправою заради вправи.

#### КРОК 1 Підготовка — створення гілки

Перед початком завжди створюйте нову гілку (правило з Модуля 1):

```
git switch main
git pull origin main
git switch -c feature/design-patterns-creational
```

#### КРОК 2 Реалізація Singleton — логер або конфігурація

Створіть файл `src/core/config.py` (або `src/core/logger.py`). Реалізуйте Singleton для конфігурації проєкту:

```
# src/core/config.py
import os

class AppConfig:
    """
    Singleton — єдиний об'єкт конфігурації застосунку.
    Зберігає налаштування, що завантажуються один раз.
    """
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance._loaded = False
        return cls._instance

    def load(self):
        """Завантажити конфігурацію (тільки один раз)"""
        if self._loaded:
            return

        # У реальному проєкті — читаємо з .env або os.environ
        self.DEBUG = os.getenv("DEBUG", "false").lower() == "true"
        self.DB_URL = os.getenv("DB_URL", "sqlite:///app.db")
        self.SECRET_KEY = os.getenv("SECRET_KEY", "dev-secret-key")
        self.LOG_LEVEL = os.getenv("LOG_LEVEL", "INFO")
        self._loaded = True
```

```

        print(f"[Config] Завантажено: DEBUG={self.DEBUG},
DB={self.DB_URL}")

    def __repr__(self):
        return f"AppConfig(debug={self.DEBUG}, db={self.DB_URL})"

# Допоміжна функція для зручного доступу
def get_config() -> AppConfig:
    cfg = AppConfig()
    cfg.load()
    return cfg

```

Перевірте роботу Singleton у файлі tests/test\_config.py:

```

from src.core.config import AppConfig, get_config

def test_singleton_same_instance():
    cfg1 = AppConfig()
    cfg2 = AppConfig()
    assert cfg1 is cfg2, "Singleton: cfg1 і cfg2 мають бути одним
об'єктом"

def test_config_loads_once():
    cfg = get_config()
    assert hasattr(cfg, 'DEBUG')
    assert hasattr(cfg, 'DB_URL')
    print("Config:", cfg)

if __name__ == "__main__":
    test_singleton_same_instance()
    test_config_loads_once()
    print("Усі перевірки пройдено!")

```

### КРОК 3 Реалізація Factory Method — фабрика для вашого проєкту

Подумайте, де у вашому проєкті потрібно створювати різні об'єкти одного типу залежно від умов. Приклади:

- Task Manager: NotifierFactory — email/sms/telegram нотифікатор
- Library: ExportFactory — PDF/Excel/JSON звіт
- Expense Tracker: PaymentFactory — cash/card/crypto платіж

Реалізуйте фабрику для ВАШОГО проєкту (мінімум 3 конкретних класи):

```

# src/services/notifier_factory.py (адаптуйте під ваш проєкт)
from abc import ABC, abstractmethod

```

```

from src.core.config import get_config

class BaseNotifier(ABC):
    @abstractmethod
    def send(self, recipient: str, message: str) -> bool:
        """Відправити повідомлення. Повертає True якщо успішно."""
        pass

    def notify_task_done(self, recipient: str, task_title: str) -> bool:
        """Шаблонний метод — однаковий для всіх нотифікаторів"""
        msg = f"Задача '{task_title}' виконана! Молодець!"
        return self.send(recipient, msg)

class EmailNotifier(BaseNotifier):
    def send(self, recipient: str, message: str) -> bool:
        config = get_config()
        if config.DEBUG:
            print(f"[DEBUG] EMAIL -> {recipient}: {message}")
            return True
        # TODO: реальна відправка через SMTP або SendGrid API
        return True

class TelegramNotifier(BaseNotifier):
    def send(self, recipient: str, message: str) -> bool:
        config = get_config()
        if config.DEBUG:
            print(f"[DEBUG] TELEGRAM -> {recipient}: {message}")
            return True
        return True

class ConsoleNotifier(BaseNotifier):
    """Для розробки та тестування"""
    def send(self, recipient: str, message: str) -> bool:
        print(f"[NOTIFY] {recipient}: {message}")
        return True

class NotifierFactory:
    """
    Factory Method: створює потрібний нотифікатор
    залежно від конфігурації або параметра.
    """
    _registry: dict = {
        "email": EmailNotifier,
        "telegram": TelegramNotifier,
        "console": ConsoleNotifier,
    }

```

```

    }

    @classmethod
    def create(cls, channel: str) -> BaseNotifier:
        cls_ = cls._registry.get(channel.lower())
        if cls_ is None:
            available = list(cls._registry.keys())
            raise ValueError(
                f"Невідомий канал '{channel}'. "
                f"Доступні: {available}"
            )
        return cls_()

    @classmethod
    def register(cls, name: str, notifier_cls):
        """Розширення без зміни фабрики (принцип ОСР)"""
        cls._registry[name] = notifier_cls

```

#### КРОК 4 Реалізація Builder — для складного об'єкта вашого проєкту

Знайдіть у своєму проєкті клас або функцію з 4+ параметрами. Типові кандидати: фільтр пошуку, HTTP-відповідь, конфігурація звіту, об'єкт задачі з опціями. Приклад для Task Manager:

```

# src/models/task_builder.py
from dataclasses import dataclass, field
from datetime import date
from typing import Optional, List

@dataclass
class Task:
    title: str
    description: str = ""
    status: str = "todo"
    priority: str = "medium"
    deadline: Optional[date] = None
    tags: List[str] = field(default_factory=list)
    assignee_id: Optional[int] = None

    def __repr__(self):
        return (f"Task('{self.title}' | {self.status} | "
                f"priority={self.priority} | deadline={self.deadline})")

class TaskBuilder:
    """
    Builder для задачі.
    Кожен метод повертає self — можна писати ланцюжком.
    """
    def __init__(self, title: str):

```

```

        if not title or not title.strip():
            raise ValueError("Заголовок задачі не може бути порожнім")
        self._task = Task(title=title.strip())

    def description(self, text: str):
        self._task.description = text
        return self

    def status(self, s: str):
        allowed = {"todo", "in_progress", "done", "cancelled"}
        if s not in allowed:
            raise ValueError(f"Статус '{s}' неприпустимий. Доступні: {allowed}")
        self._task.status = s
        return self

    def priority(self, p: str):
        allowed = {"low", "medium", "high", "critical"}
        if p not in allowed:
            raise ValueError(f"Пріоритет '{p}' неприпустимий.")
        self._task.priority = p
        return self

    def deadline(self, d: date):
        if d < date.today():
            raise ValueError("Дедлайн не може бути в минулому")
        self._task.deadline = d
        return self

    def tag(self, t: str):
        if t not in self._task.tags:
            self._task.tags.append(t.lower())
        return self

    def assign_to(self, user_id: int):
        self._task.assignee_id = user_id
        return self

    def build(self) -> Task:
        return self._task

# Демонстрація — читається як природна мова:
from datetime import timedelta

task = (
    TaskBuilder("Реалізувати сторінку авторизації")
    .description("Форма входу з JWT токенами, валідація email")
    .priority("high")

```

```

        .deadline(date.today() + timedelta(days=7))
        .tag("frontend")
        .tag("auth")
        .assign_to(user_id=42)
        .build()
    )
    print(task)

```

## КРОК 5 Написати демо-скрипт

Створіть файл `demo/patterns_demo.py`, який демонструє всі три патерни разом:

```

# demo/patterns_demo.py
"""
Демонстрація породжуючих патернів у контексті проєкту.
Запуск: python demo/patterns_demo.py
"""

print("=" * 50)
print("ПАТЕРНИ ПРОЕКТУВАННЯ — ДЕМОНСТРАЦІЯ")
print("=" * 50)

# 1. Singleton
print("
--- Singleton: AppConfig ---")
from src.core.config import get_config
cfg1 = get_config()
cfg2 = get_config()
print(f"cfg1 is cfg2: {cfg1 is cfg2}") # True

# 2. Factory Method
print("
--- Factory Method: NotifierFactory ---")
from src.services.notifier_factory import NotifierFactory
for channel in ["email", "telegram", "console"]:
    n = NotifierFactory.create(channel)
    n.send("test@user.com", f"Тест через {channel}")

# 3. Builder
print("
--- Builder: TaskBuilder ---")
from src.models.task_builder import TaskBuilder
from datetime import date, timedelta
task = (TaskBuilder("Навчитись патернам")
        .priority("high")
        .tag("learning")
        .deadline(date.today() + timedelta(days=3))
        .build())
print(task)

```

## КРОК 6 Коміт та Pull Request

```
git add src/ demo/ tests/
```

```
git commit -m "feat: додати породжуючі патерни Singleton, Factory, Builder"
git push origin feature/design-patterns-creational
# Далі: створити PR на GitHub (як у Модулі 1, ПР-3)
```

## РЕЗУЛЬТАТ Очікуваний результат роботи

Артефакти для задачі:

- `src/core/config.py` — Singleton конфігурації з тестом
- `src/services/notifier_factory.py` (або аналог) — Factory Method з 3+ класами
- `src/models/task_builder.py` (або аналог) — Builder з валідацією
- `demo/patterns_demo.py` — демо-скрипт з виводом у термінал
- Закритий Pull Request у GitHub із linked Issue

### ЗАВДАННЯ ДЛЯ ЗАКРІПЛЕННЯ 8.1 Singleton — глибше розуміння

Завдання А: Реалізуйте Thread-safe Singleton для Python (підказка: використовуйте `threading.Lock()` щоб уникнути race condition при одночасному доступі з кількох потоків).

Завдання В: Поясніть письмово (3-5 речень): чому наступний код НЕ є Singleton, хоча виглядає схоже?

```
class Config:
    data = {} # змінна класу — спільна для всіх екземплярів

    def set(self, key, value):
        Config.data[key] = value

c1 = Config()
c2 = Config()
print(c1 is c2) # False — це різні об'єкти, хоча data спільна
```

### ЗАВДАННЯ ДЛЯ ЗАКРІПЛЕННЯ 8.2 Factory Method — розширення

Додайте до вашої фабрики новий тип нотифікатора (або експортера, або будь-якого типу з вашого проекту) без зміни жодного рядка в `NotifierFactory` та клієнтському коді. Використайте метод `register()`.

Підказка: `NotifierFactory.register('slack', SlackNotifier)` — і новий клас автоматично доступний через `create('slack')`.

### ? Запитання для самоперевірки

1. Яку проблему вирішує Factory Method? Наведіть ситуацію з вашого проєкту, де без цього патерну код ставав би гіршим.
2. У чому різниця між Builder та конструктором з багатьма параметрами? Коли Builder кращий?
3. Пояснить рядок `'return self'` у методах Builder. Що він дає? Як називається ця техніка?
4. Singleton зберігає стан між викликами. Чому це ускладнює тестування? Як можна написати тест, що не залежить від попереднього?



## 1. Теоретична частина

### 1.1. Патерн Observer — підписка на події

Observer (спостерігач) визначає залежність типу «один до багатьох» між об'єктами: при зміні стану одного об'єкта всі залежні об'єкти отримують повідомлення та автоматично оновлюються. Це основа event-driven архітектури.

#### АНАЛОГІЯ Аналогія з реального життя

Уявіть YouTube-канал. Автор (Subject) публікує відео. Підписники (Observers) автоматично отримують сповіщення. Автор не знає конкретних підписників — просто 'розсилає' подію. Нові підписники підписуються самостійно. Відписка теж без участі автора. Observer — це і є механізм підписки в коді.

```
# Python: Observer для системи задач
from abc import ABC, abstractmethod
from typing import List

class TaskObserver(ABC):
    """Інтерфейс спостерігача"""
    @abstractmethod
    def on_task_updated(self, task_id: int, event: str, data: dict):
        pass

class Task:
    """Subject (суб'єкт) — той, за ким спостерігають"""
    def __init__(self, task_id: int, title: str):
        self.id = task_id
        self.title = title
        self.status = "todo"
        self._observers: List[TaskObserver] = []

    def subscribe(self, observer: TaskObserver):
        """Додати підписника"""
        self._observers.append(observer)
        print(f"[Observer] {type(observer).__name__} підписався на задачу #{self.id}")

    def unsubscribe(self, observer: TaskObserver):
        """Відписати спостерігача"""
        self._observers.remove(observer)
```

```

def _notify(self, event: str, data: dict = None):
    """Приватний: сповістити всіх підписників"""
    for obs in self._observers:
        obs.on_task_updated(self.id, event, data or {})

def complete(self):
    self.status = "done"
    self._notify("completed", {"title": self.title})

def assign(self, user_id: int):
    self._notify("assigned", {"user_id": user_id, "title":
self.title})

# Конкретні спостерігачі
class EmailNotificationObserver(TaskObserver):
    def on_task_updated(self, task_id, event, data):
        if event == "completed":
            print(f"[EMAIL] Задача #{task_id} '{data['title']}'
виконана!")

class ActivityLogObserver(TaskObserver):
    def __init__(self):
        self.log = []
    def on_task_updated(self, task_id, event, data):
        entry = f"Task #{task_id}: {event} - {data}"
        self.log.append(entry)
        print(f"[LOG] {entry}")

class StatisticsObserver(TaskObserver):
    def __init__(self):
        self.completed = 0
    def on_task_updated(self, task_id, event, data):
        if event == "completed":
            self.completed += 1
            print(f"[STATS] Всього виконано задач: {self.completed}")

# Використання
task = Task(1, "Написати документацію")
stats = StatisticsObserver()
task.subscribe(EmailNotificationObserver())
task.subscribe(ActivityLogObserver())
task.subscribe(stats)

task.assign(user_id=5)      # спрацьовують ActivityLog
task.complete()            # спрацьовують всі три

```

## ІНДУСТРІЯ Як це використовують у роботі

Observer — основа фреймворків: `addEventListener` у браузері — Observer. Django Signals (`post_save`, `pre_delete`) — Observer. React `useEffect` залежності — Observer. Kafka/RabbitMQ — Observer у розподілених системах. Node.js `EventEmitter` — Observer. Це один із найважливіших патернів у сучасній розробці.

### 1.2. Патерн Strategy — замінний алгоритм

Strategy (стратегія) визначає сімейство алгоритмів, інкапсулює кожен з них і робить їх взаємозамінними. Стратегія дозволяє змінювати алгоритми незалежно від клієнтів, які ними користуються.

## АНАЛОГІЯ Аналогія з реального життя

Уявіть GPS-навігатор. Є три стратегії маршруту: найкоротший шлях, найшвидший шлях, уникаючи платних доріг. Ви переключаєтеся між ними натисканням кнопки — навігатор той самий, змінюється лише алгоритм прокладення маршруту. Strategy — саме це: один контекст, різні алгоритми.

```
# Python: Strategy для сортування задач
from abc import ABC, abstractmethod
from typing import List

class SortStrategy(ABC):
    @abstractmethod
    def sort(self, tasks: List) -> List:
        pass

class SortByDeadline(SortStrategy):
    def sort(self, tasks: List) -> List:
        return sorted(tasks, key=lambda t: (t["deadline"] is None,
        t["deadline"]))

class SortByPriority(SortStrategy):
    PRIORITY_ORDER = {"critical": 0, "high": 1, "medium": 2, "low": 3}
    def sort(self, tasks: List) -> List:
        return sorted(tasks, key=lambda t:
        self.PRIORITY_ORDER.get(t["priority"], 99))

class SortByCreatedAt(SortStrategy):
    def sort(self, tasks: List) -> List:
        return sorted(tasks, key=lambda t: t["created_at"], reverse=True)
```

```

class TaskList:
    """Контекст — використовує стратегію"""
    def __init__(self, strategy: SortStrategy = None):
        self._strategy = strategy or SortByCreatedAt()
        self._tasks = []

    def set_strategy(self, strategy: SortStrategy):
        """Змінити стратегію у runtime"""
        self._strategy = strategy

    def add(self, task: dict):
        self._tasks.append(task)

    def get_sorted(self) -> list:
        return self._strategy.sort(self._tasks)

# Демонстрація
from datetime import date
tasks = TaskList()
tasks.add({"title": "Задача А", "priority": "low", "deadline":
date(2025,3,1), "created_at": 1})
tasks.add({"title": "Задача В", "priority": "critical", "deadline": None,
"created_at": 2})
tasks.add({"title": "Задача С", "priority": "high", "deadline":
date(2025,2,1), "created_at": 3})

print("За дедлайном:", [t["title"] for t in
TaskList(SortByDeadline()).get_sorted() if tasks._tasks.append(t) or
True][0:0] or
      [t["title"] for t in SortByDeadline().sort(tasks._tasks)])
print("За пріоритетом:", [t["title"] for t in
SortByPriority().sort(tasks._tasks)])

```

### 1.3. Патерн Decorator — динамічне додавання поведінки

Decorator (декоратор) динамічно додає нову відповідальність об'єкту. Декоратори надають гнучку альтернативу породженню підкласів для розширення функціональності.

#### АНАЛОГІЯ Аналогія з реального життя

Уявіть звичайну каву. Хочете молоко — додали молоко (Decorator). Хочете цукор — додали цукор (ще один Decorator). Хочете корицю — ще один. Кожна добавка загортає попередню і розширює функціональність. Кава лишається кавою, але стає складнішою. Ціна — сума всіх компонентів.

```

# Python: Decorator для кешування та логування функцій
import time
import functools
from typing import Callable

# Decorator як функція (Python-стиль, найпоширеніший)
def timer(func: Callable) -> Callable:
    """Вимірює час виконання функції"""
    @functools.wraps(func) # зберігає ім'я та docstring оригіналу
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        elapsed = time.perf_counter() - start
        print(f"[TIMER] {func.__name__} виконувалась {elapsed:.4f} сек")
        return result
    return wrapper

def cache(func: Callable) -> Callable:
    """Простий кеш: зберігає результат для повторних викликів"""
    _cache = {}
    @functools.wraps(func)
    def wrapper(*args):
        if args not in _cache:
            _cache[args] = func(*args)
            print(f"[CACHE] Обчислено для {args}")
        else:
            print(f"[CACHE] З кешу для {args}")
        return _cache[args]
    return wrapper

def log_call(func: Callable) -> Callable:
    """Логує кожен виклик функції"""
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        print(f"[LOG] Виклик {func.__name__} (args={args})")
        result = func(*args, **kwargs)
        print(f"[LOG] {func.__name__} повернула {result!r}")
        return result
    return wrapper

# Застосування декораторів (порядок знизу вгору):
@timer          # 3. виміряти час
@log_call       # 2. залогувати виклик
@cache          # 1. перевірити кеш
def get_user_tasks(user_id: int) -> list:
    """Отримати задачі користувача з БД (імітація)"""
    time.sleep(0.1) # імітація запиту до БД
    return [{"id": 1, "title": f"Задача user {user_id}"}]

# Перший виклик — обчислення + лог + таймер
tasks1 = get_user_tasks(42)

```

```
# Другий виклик — з кешу (без sleep)
tasks2 = get_user_tasks(42)
# Пізній user_id — знову обчислення
tasks3 = get_user_tasks(99)
```

#### 1.4. Патерн Facade — простий інтерфейс до складної підсистеми

Facade (фасад) надає спрощений інтерфейс до складної підсистеми з багатьма класами. Клієнт взаємодіє з фасадом замість прямого використання десятків класів.

##### АНАЛОГІЯ Аналогія з реального життя

Пульт від телевізора — це Facade. Натиснули 'Увімкнути' — і телевізор увімкнувся, підключився до мережі, завантажив канали, налаштував звук. Насправді відбулося 20 різних операцій. Але вам не потрібно знати про кожну з них. Facade — це пульт для вашої підсистеми.

```
# Python: Facade для відправки задач через кілька сервісів
class UserRepository:
    def get_by_id(self, user_id: int) -> dict:
        return {"id": user_id, "email": "user@example.com", "name": "Іван"}

class TaskRepository:
    def create(self, data: dict) -> dict:
        return {**data, "id": 101}

class EmailService:
    def send(self, to: str, subject: str, body: str):
        print(f"[EMAIL] To: {to} | {subject}")

class NotificationService:
    def push(self, user_id: int, message: str):
        print(f"[PUSH] User {user_id}: {message}")

class ActivityLogger:
    def log(self, user_id: int, action: str):
        print(f"[ACTIVITY] User {user_id}: {action}")

# — FACADE: один метод замість 5 різних сервісів —
class TaskFacade:
    """
    Фасад для операцій з задачами.
    Клієнт не знає про UserRepository, EmailService тощо.
    """
```

```

def __init__(self):
    self._users = UserRepository()
    self._tasks = TaskRepository()
    self._email = EmailService()
    self._notif = NotificationService()
    self._activity = ActivityLogger()

def create_task_for_user(self, user_id: int, task_data: dict) -> dict:
    """
    Складна операція за лаштунками:
    1. Отримати користувача
    2. Створити задачу
    3. Відправити email
    4. Надіслати push
    5. Залогувати дію
    """
    user = self._users.get_by_id(user_id)
    task = self._tasks.create(**task_data, "user_id": user_id)

    self._email.send(
        to=user["email"],
        subject="Нова задача призначена вам",
        body=f"Задача: {task_data['title']}"
    )
    self._notif.push(user_id, f"Нова задача: {task_data['title']}")
    self._activity.log(user_id, f"Створено задачу #{task['id']}")

    return task

# Клієнтський код — простий і зрозумілий
facade = TaskFacade()
result = facade.create_task_for_user(
    user_id=42,
    task_data={"title": "Підготувати звіт", "priority": "high"}
)
print(f"Створено задачу: {result}")

```

### ПАМ'ЯТКА Коротко і по суті

- Observer: 'підпишись і отримуй сповіщення' — для подій, логів, статистики
- Strategy: 'вибери алгоритм у runtime' — для сортування, платежів, маршрутів
- Decorator: 'загорни і додай поведінку' — для кешу, логу, авторизації
- Facade: 'один метод замість п'яти' — для спрощення складних підсистем

## 2. Практичне завдання ПР-9

### ПРАКТИЧНЕ ЗАВДАННЯ 9.1 Реалізація поведінкових та структурних патернів

#### КРОК 1 Підготовка — нова гілка

```
git switch main && git pull origin main  
git switch -c feature/design-patterns-behavioral
```

#### КРОК 2 Observer — система подій вашого проєкту

Визначте, які події відбуваються у вашому проєкті (task.completed, user.registered, order.placed тощо). Реалізуйте Observer для мінімум двох різних подій:

- Мінімум 3 класи-спостерігачі (логер, нотифікатор, статистика або аналог)
- Subject сповіщає підписників автоматично при зміні стану
- Підписка і відписка через методи subscribe/unsubscribe

#### КРОК 3 Strategy — логіка, що змінюється

Знайдіть у своєму проєкті місце, де логіка може варіюватися. Приклади:

- Task Manager: стратегії сортування (за дедлайном, пріоритетом, датою)
- Library: стратегії пошуку (за назвою, за автором, за ISBN)
- Expense Tracker: стратегії розрахунку звіту (за місяць, за категорією)

Реалізуйте мінімум 3 конкретні стратегії. Покажіть, що стратегію можна замінити без зміни контексту.

#### КРОК 4 Decorator — кешування або логування

Реалізуйте мінімум 2 декоратори для функцій або методів вашого проєкту:

- Обов'язковий: @timer — вимірювання часу виконання (з шаблону вище)
- Обов'язковий: @cache або @log\_call
- Бонус: @retry(times=3) — перезапуск функції при помилці

Застосуйте декоратори до реальних функцій вашого проєкту (наприклад, функція запити до БД).

#### КРОК 5 Facade — спрощуємо роботу з модулями



Визначте найскладнішу операцію у вашому проєкті (яка зачіпає 3+ сервіси). Реалізуйте для неї Facade. Мінімальна вимога: Facade-метод всередині викликає мінімум 3 різні сервіси.

#### КРОК 6 Тест-демонстрація патернів

Додайте до `demo/patterns_demo.py` демонстрацію нових патернів з виводом у термінал. Запустіть — переконайтесь що все працює.

#### КРОК 7 Коміт та PR

```
git add src/ demo/  
git commit -m "feat: додати поведінкові патерни Observer, Strategy,  
Decorator, Facade"  
git push origin feature/design-patterns-behavioral
```

### РЕЗУЛЬТАТ Очікуваний результат роботи

Артефакти для здачі:

- Observer: Subject клас + мінімум 3 Observer класи
- Strategy: мінімум 3 конкретні стратегії + Context клас
- Decorator: мінімум 2 декоратори, застосованих до реального коду
- Facade: один клас, що спрощує роботу з 3+ підсистемами
- Демо-скрипт з виводом у термінал + Закритий PR

#### ЗАВДАННЯ ДЛЯ ЗАКРІПЛЕННЯ 9.1 Observer — глибше

Реалізуйте простий EventEmitter (як у Node.js): клас `EventBus` з методами `on(event, handler)` і `emit(event, data)`. Підпишіть 3 різні обробники на одну подію `'task:completed'`. Перевірте, що при `emit` всі три спрацьовують.

#### ЗАВДАННЯ ДЛЯ ЗАКРІПЛЕННЯ 9.2 Strategy + Factory разом

Створіть `SortStrategyFactory`, який повертає потрібну стратегію за рядком: `SortStrategyFactory.create('deadline') → SortByDeadline()`. Це поєднання двох патернів — часта практика у реальному кодi.

### ? Запитання для самоперевірки

1. Яка принципова різниця між Observer і простим викликом функцій callbacks? У чому перевага Observer для великих систем?
2. Як Strategy дотримується принципу Open/Closed (ОСР) з Модуля 2? Поясніть на прикладі додавання нового алгоритму сортування.
3. Чому Decorator зручніший за успадкування для додавання поведінки? Наведіть приклад ситуації, де Decorator вигравав би.
4. Де у вашому проєкті Facade вже існує неявно? Знайдіть місце, де один метод приховує кілька операцій.

## 1. Теоретична частина

### 1.1. Принципи надійного коду

### 1.2. Ієрархія власних виключень

Замість генерування вбудованих `Exception` або `ValueError` для будь-якої помилки — створюйте власну ієрархію виключень. Це дозволяє ловити помилки точно, а не «ловити все підряд».

```
# src/exceptions.py — власна ієрархія виключень

class AppError(Exception):
    """
    Базовий клас для всіх помилок застосунку.
    Дозволяє відловити БУДЬ-яку помилку додатку:
    except AppError as e: ...
    """
    def __init__(self, message: str, code: str = "APP_ERROR"):
        super().__init__(message)
        self.message = message
        self.code = code      # машинночитаемый код для API

    def to_dict(self) -> dict:
        return {"error": self.code, "message": self.message}

# Група: помилки валідації
class ValidationError(AppError):
    """Вхідні дані не пройшли валідацію"""
    def __init__(self, field: str, message: str):
        super().__init__(f"Поле '{field}': {message}", "VALIDATION_ERROR")
        self.field = field

# Група: помилки бізнес-логіки
class BusinessError(AppError):
    """Порушення бізнес-правила"""
    pass

class TaskNotFoundError(BusinessError):
    def __init__(self, task_id: int):
        super().__init__(f"Задачу #{task_id} не знайдено",
                        "TASK_NOT_FOUND")
        self.task_id = task_id
```

```

class TaskAlreadyDoneError(BusinessError):
    def __init__(self, task_id: int):
        super().__init__(f"Задача #{task_id} вже виконана",
            "TASK_ALREADY_DONE")

# Група: інфраструктурні помилки
class InfrastructureError(AppError):
    """Зовнішні залежності недоступні"""
    pass

class DatabaseError(InfrastructureError):
    def __init__(self, operation: str, detail: str):
        super().__init__(f"БД: операція '{operation}' не вдалась:
{detail}", "DB_ERROR")

# Приклад використання:
def complete_task(task_id: int, current_user_id: int):
    task = get_task(task_id)          # може кинути TaskNotFoundError
    if task is None:
        raise TaskNotFoundError(task_id)
    if task["status"] == "done":
        raise TaskAlreadyDoneError(task_id)
    if task["user_id"] != current_user_id:
        raise BusinessError("Не можна завершити чужу задачу", "FORBIDDEN")
    # ... завершення задачі

```

### 1.3. Стратегії try/except — правильне використання

Блок try/except потрібен там, де може виникнути помилка, яку можна обробити. Але невірне використання — одна з найпоширеніших помилок початківців:

| Антипатерн                        | Чому погано                            | Як правильно                                     |
|-----------------------------------|----------------------------------------|--------------------------------------------------|
| <b>except: pass</b>               | Помилка «зникає» мовчки                | Мінімум: логувати помилку перед pass             |
| <b>except Exception as e: ...</b> | Ловить все, включаючи системні помилки | Ловити конкретний тип: except ValueError         |
| <b>try/except навколо всього</b>  | Приховує де саме виникла помилка       | Мінімальний блок try — лише небезпечний код      |
| <b>raise Exception('помилка')</b> | Загальне виключення без контексту      | Кидати власний клас: raise TaskNotFoundError(id) |

```

# Правильне використання try/except

import logging
logger = logging.getLogger(__name__)

def get_task_safe(task_id: int) -> dict | None:
    """
    Безпечне отримання задачі.
    Повертає None якщо не знайдена, кидає виключення при помилці БД.
    """
    try:
        # Тільки операція, що може впасти — в блоці try
        task = database.get("tasks", task_id)

    except DatabaseError as e:
        # Інфраструктурна помилка — логуємо і пробрасуємо
        logger.error(f"БД недоступна при отриманні задачі #{task_id}: {e}")
        raise # пробрасуємо далі — не ковтаємо

    except Exception as e:
        # Несподівана помилка — логуємо з traceback
        logger.exception(f"Несподівана помилка в get_task({task_id})")
        raise InfrastructureError("database", str(e))

    else:
        # Виконується якщо try пройшов без помилок
        logger.debug(f"Задача #{task_id} успішно отримана")
        return task

    # finally — виконується завжди (закрити з'єднання, очистити ресурс)
    # finally:
    #     database.release_connection()

```

## 1.4. Валідація вхідних даних

Валідація — це перевірка, що вхідні дані відповідають очікуваним вимогам до того, як вони потраплять у бізнес-логіку. Правило: валідуйте на межі системи — у місці, де дані входять (HTTP endpoint, CLI-аргумент, читання файлу).

```

# Python: Валідація без зовнішніх бібліотек
import re
from datetime import date

class TaskValidator:
    """
    Валідатор даних задачі.
    Викидає ValidationError при першій знайденій помилці.

```

```

Або збирає всі помилки — залежно від потреби.
"""

ALLOWED_STATUSES = {"todo", "in_progress", "done", "cancelled"}
ALLOWED_PRIORITIES = {"low", "medium", "high", "critical"}
MAX_TITLE_LENGTH = 200
MIN_TITLE_LENGTH = 3

@classmethod
def validate_create(cls, data: dict) -> dict:
    """
    Валідувати дані для створення задачі.
    Повертає очищені (sanitized) дані.
    Кидає ValidationError при помилці.
    """
    errors = []

    # Перевірка title
    title = data.get("title", "")
    if not isinstance(title, str):
        errors.append(("title", "Має бути рядком"))
    elif len(title.strip()) < cls.MIN_TITLE_LENGTH:
        errors.append(("title", f"Мінімум {cls.MIN_TITLE_LENGTH} СИМВОЛИ"))
    elif len(title) > cls.MAX_TITLE_LENGTH:
        errors.append(("title", f"Максимум {cls.MAX_TITLE_LENGTH} СИМВОЛІВ"))

    # Перевірка priority (необов'язкове поле)
    priority = data.get("priority", "medium")
    if priority not in cls.ALLOWED_PRIORITIES:
        errors.append(("priority", f"Допустимі значення: {cls.ALLOWED_PRIORITIES}"))

    # Перевірка deadline (необов'язкове поле)
    if "deadline" in data and data["deadline"] is not None:
        try:
            if isinstance(data["deadline"], str):
                deadline = date.fromisoformat(data["deadline"])
            else:
                deadline = data["deadline"]
            if deadline < date.today():
                errors.append(("deadline", "Не може бути в минулому"))
        except ValueError:
            errors.append(("deadline", "Формат: YYYY-MM-DD"))

    # Якщо є помилки — кидаємо зі списком
    if errors:
        field, msg = errors[0] # перша помилка
        raise ValidationError(field, msg)

```

```
# Повертаємо очищені дані
return {
    "title": title.strip(),
    "priority": priority,
    "deadline": data.get("deadline"),
    "description": data.get("description", "").strip(),
}
```

## 1.5. Логування — журнал подій системи

Логування (logging) — запис подій, що відбуваються в системі під час виконання. Логи — це очі розробника у production-системі, де debugger недоступний.

| Рівень логу     | Коли використовувати та приклади                                                                                      |
|-----------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>DEBUG</b>    | Детальна діагностична інформація. Лише при розробці. SQL-запити, значення змінних, трасування потоку виконання        |
| <b>INFO</b>     | Підтвердження нормальної роботи. Запити до API, успішні операції, завантаження конфігурації                           |
| <b>WARNING</b>  | Щось неочікуване, але програма продовжує. Застарілий API, повільний запит, нестандартний стан                         |
| <b>ERROR</b>    | Серйозна помилка — функція не виконана. Виключення, збій зовнішнього сервісу, нульовий результат там де не очікувався |
| <b>CRITICAL</b> | Системна помилка — застосунок може зупинитись. БД недоступна, вичерпана пам'ять, критична помилка безпеки             |

```
# src/core/logging_setup.py — налаштування логування
import logging
import json
from datetime import datetime

class JsonFormatter(logging.Formatter):
    """
    Структурований JSON-формат логів.
    Зручно для Grafana, Datadog, ELK Stack.
    """
    def format(self, record: logging.LogRecord) -> str:
        log_entry = {
            "timestamp": datetime.utcnow().isoformat() + "Z",
            "level": record.levelname,
            "logger": record.name,
            "message": record.getMessage(),
            "module": record.module,
            "line": record.lineno,
        }
        # Додати exc_info якщо є виключення
```

```

        if record.exc_info:
            log_entry["exception"] = self.formatException(record.exc_info)
        return json.dumps(log_entry, ensure_ascii=False)

def setup_logging(level: str = "INFO", json_format: bool = False):
    """
    Налаштувати логування для всього застосунку.
    Викликається один раз при старті.
    """
    root_logger = logging.getLogger()
    root_logger.setLevel(getattr(logging, level.upper(), logging.INFO))

    # Видалити існуючі handlers
    root_logger.handlers.clear()

    # Handler для виводу в консоль
    handler = logging.StreamHandler()
    if json_format:
        handler.setFormatter(JsonFormatter())
    else:
        fmt = "%(asctime)s [%(levelname)-8s] %(name)s: %(message)s"
        handler.setFormatter(logging.Formatter(fmt, datefmt="%H:%M:%S"))

    root_logger.addHandler(handler)

# Використання в модулях:
# import logging
# logger = logging.getLogger(__name__) # ім'я модуля як ім'я логера
#
# logger.debug("Запит до БД: SELECT * FROM tasks WHERE user_id=%s",
# user_id)
# logger.info("Задача #%d успішно створена", task.id)
# logger.warning("Запит виконувався %d мс — повільніше за норму",
# elapsed_ms)
# logger.error("Не вдалось відправити email: %s", str(e))
# logger.exception("Несподівана помилка") # додає traceback автоматично

```

## ІНДУСТРІЯ Як це використовують у роботі

Структуровані JSON-логи — стандарт у хмарних системах (AWS CloudWatch, Google Cloud Logging, Datadog). Кожен лог — це JSON-об'єкт, а не рядок тексту. Це дозволяє фільтрувати логи за будь-яким полем: 'покажи всі ERROR від user\_id=42 за останню годину'. Звичайний текстовий лог такого не підтримує.



## 2. Практичне завдання ПР-10

### ПРАКТИЧНЕ ЗАВДАННЯ 10.1 Обробка помилок, власні виключення та логування в проєкті

#### КРОК 1 Нова гілка

```
git switch -c feature/error-handling-and-logging
```

#### КРОК 2 Створити ієрархію виключень

Створіть файл `src/exceptions.py`. Реалізуйте мінімум:

- `AppError` (базовий)
- `ValidationError` (з полем `field`)
- Мінімум 3 бізнес-помилки для вашого проєкту (`TaskNotFoundError` або аналог)
- Метод `to_dict()` у базовому класі (для повернення в API)

Переконайтесь, що кожен клас описує конкретну ситуацію, а не є копією базового.

#### КРОК 3 Налаштувати логування

Створіть `src/core/logging_setup.py` з функцією `setup_logging()`. Підтримуйте два режими:

- Режим `DEBUG` (розробка): текстовий формат, рівень `DEBUG`
- Режим `PRODUCTION`: JSON-формат, рівень `WARNING`

Додайте виклик `setup_logging()` на початку `main.py` або `app.py` вашого проєкту.

#### КРОК 4 Додати логування у існуючі сервіси

Відкрийте будь-який файл сервісу з Модуля 2 або ПР-8. Додайте логування:

- `INFO`: початок і кінець важливих операцій
- `DEBUG`: деталі (параметри запиту, проміжні результати)
- `WARNING`: нестандартні ситуації (запит повернув порожній список)
- `ERROR`: у блоках `except`

Правило: мінімум 5 рядків логування у сервісі.

#### КРОК 5 Реалізувати валідатор для вашого проєкту

Створіть `src/validators/task_validator.py` (або аналог). Реалізуйте:

- Перевірку обов'язкових полів
- Перевірку довжини рядків
- Перевірку допустимих значень (enum полів)
- Перевірку формату дати (якщо є)

Інтегруйте валідатор у відповідний сервіс: перед збереженням викликати `validate()`.

#### КРОК 6 Написати тест для обробки помилок

Створіть `tests/test_exceptions.py`:

```
from src.exceptions import ValidationError, TaskNotFoundError

def test_validation_error_has_field():
    err = ValidationError("title", "занадто короткий")
    assert err.field == "title"
    assert "title" in str(err)

def test_task_not_found_dict():
    err = TaskNotFoundError(99)
    d = err.to_dict()
    assert d["error"] == "TASK_NOT_FOUND"
    assert "99" in d["message"]

def test_business_error_hierarchy():
    from src.exceptions import AppError
    err = TaskNotFoundError(1)
    assert isinstance(err, AppError)  # ієрархія працює

if __name__ == "__main__":
    test_validation_error_has_field()
    test_task_not_found_dict()
    test_business_error_hierarchy()
    print("Всі тести пройдено!")
```

#### КРОК 7 Коміт та PR

```
git add src/exceptions.py src/core/logging_setup.py src/validators/ tests/
git commit -m "feat: додати власні виключення, логування та валідацію"
git push origin feature/error-handling-and-logging
```

## РЕЗУЛЬТАТ Очікуваний результат роботи

Артефакти для задачі:

- `src/exceptions.py` — ієрархія з 5+ класами та `to_dict()`
- `src/core/logging_setup.py` — налаштування JSON і текстового формату
- Мінімум один сервіс з 5+ рядками логування
- `src/validators/` — валідатор для основного ресурсу
- `tests/test_exceptions.py` — 3+ тести проходять

## ЗАВДАННЯ ДЛЯ ЗАКРІПЛЕННЯ 10.1 Retry decorator

Реалізуйте декоратор `@retry(max_attempts=3, delay=0.5)`, який перезапускає функцію при виключенні певного типу. Застосуйте до функції, що імітує ненадійне з'єднання (випадково кидає `ConnectionError` з ймовірністю 50%).

```
import random, time, functools

def retry(max_attempts=3, delay=0.5, exceptions=(Exception,)):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            # Ваша реалізація тут
            ...
        return wrapper
    return decorator

@retry(max_attempts=3, delay=0.1, exceptions=(ConnectionError,))
def unstable_request(url: str) -> dict:
    if random.random() < 0.5:
        raise ConnectionError("Тимчасово недоступно")
    return {"status": "ok", "url": url}
```

## ? Запитання для самоперевірки

1. Чому 'except Exception: pass' — небезпечний антипатерн? Що може статись у production якщо він залишиться в коді?
2. Яка різниця між `raise` та `raise e` у блоці `except`? Коли кожен варіант кращий?

3. Навіщо структуровані JSON-логи кращі за звичайні текстові у production-системі? Наведіть конкретний приклад запиту до логів.

4. Де у вашому проєкті валідація найважливіша? Що може статись якщо її не робити?

## 1. Теоретична частина

### 1.1. Навіщо Code Review

Code Review (рев'ю коду, перевірка коду) — процес, коли інший розробник читає ваш код до того, як він потрапляє в основну гілку. Це не контроль і не критика автора — це спільна відповідальність за якість коду.

#### АНАЛОГІЯ Аналогія з реального життя

Уявіть видавця книг. Навіть найкращий письменник має редактора. Не тому що письменник поганий — а тому що свій текст читаєш очима, які знають що там написано, і не бачать помилок. Чужі очі бачать інакше. Code Review — це редактор для вашого коду.

Що знаходять на Code Review: логічні помилки (edge cases, неправильні умови), проблеми безпеки (SQL injection, відкриті ключі), порушення архітектурних рішень команди, дублювання коду, відсутні тести, незрозумілі назви змінних та функцій.

### 1.2. Як проводити Code Review правильно

| Що перевіряти  | Конкретні запитання при рев'ю                                                          |
|----------------|----------------------------------------------------------------------------------------|
| Коректність    | Чи вирішує код задачу? Чи обробляє edge cases (порожній список, None, від'ємне число)? |
| Безпека        | Чи є SQL injection, XSS, відкриті секрети, незахищені ендпоінти?                       |
| Читабельність  | Чи зрозуміле ім'я функції без читання тіла? Чи потрібні коментарі?                     |
| Тести          | Чи є тести для нової логіки? Чи покривають happy path і error cases?                   |
| Архітектура    | Чи відповідає код прийнятим рішенням (ADR з Модуля 2)?                                 |
| Продуктивність | Чи немає N+1 запитів до БД, зайвих циклів, великих in-memory операцій?                 |

## ПРИКЛАД Практичний приклад

Хороший коментар на рев'ю: 'У рядку 42, якщо список tasks порожній, sorted() поверне [], але наступний tasks[0] кине IndexError. Додай перевірку if not tasks: return None або використай tasks[0] if tasks else None.'

Поганий коментар: 'Це неправильно.' або 'Перепиши.' — без пояснення чому і як.

### 1.3. Code Smells — запах поганого коду

Code Smell — ознака в коді, що сигналізує про можливу проблему проектування. Вони не завжди є помилками, але завжди є приводом замислитись. Нижче — найпоширеніші:

| Назва               | Ознака                                                     | Як виправити                                              |
|---------------------|------------------------------------------------------------|-----------------------------------------------------------|
| Long Method         | Функція довша за 20-30 рядків — важко читати і тестувати   | Extract Function: виносити логічні блоки в окремі функції |
| Magic Number        | Число в коді без пояснення: if status == 3                 | Константа: STATUS_DONE = 3 або Enum                       |
| Duplicate Code      | Однаковий або схожий код у 2+ місцях                       | DRY: виносити в спільну функцію або базовий клас          |
| Large Class         | Клас з 500+ рядків і 20+ методами                          | Розбити на кілька класів за SRP                           |
| Feature Envy        | Метод більше звертається до іншого класу, ніж свого        | Move Method в той клас, до якого звертається              |
| Long Parameter List | Функція приймає 5+ параметрів                              | Замінити на об'єкт (Parameter Object) або Builder         |
| Comments as Excuses | Коментар пояснює що робить код, а не чому                  | Перейменувати функцію/змінну щоб код говорив сам себе     |
| Dead Code           | Код, що ніколи не виконується (commented-out, unreachable) | Видалити — він є у Git history якщо потрібен              |

### 1.4. Техніки рефакторингу Фаулера

Рефакторинг — це змінення структури коду без змінення його поведінки. Ключова умова: тести повинні залишатись зеленими до і після кожного кроку рефакторингу. Мартін Фаулер описав 72 техніки у книзі «Refactoring» (2018). Найважливіші:

```

# — Техніка 1: Extract Function —————
# BEFORE: один великий метод
def process_order(order):
    # валідація
    if not order.get("items"):
        raise ValueError("Порожнє замовлення")
    if order.get("total", 0) <= 0:
        raise ValueError("Сума <= 0")
    # розрахунок знижки
    discount = 0
    if order.get("user_type") == "vip":
        discount = 0.15
    elif order.get("total", 0) > 1000:
        discount = 0.05
    final = order["total"] * (1 - discount)
    # збереження
    db.save("orders", {**order, "discount": discount, "final": final})
    return final

# AFTER: виносимо у окремі функції
def validate_order(order: dict):
    if not order.get("items"):
        raise ValidationError("items", "Порожнє замовлення")
    if order.get("total", 0) <= 0:
        raise ValidationError("total", "Сума має бути > 0")

def calculate_discount(order: dict) -> float:
    if order.get("user_type") == "vip":
        return 0.15
    if order.get("total", 0) > 1000:
        return 0.05
    return 0.0

def process_order(order: dict) -> float:
    validate_order(order)
    discount = calculate_discount(order)
    final = order["total"] * (1 - discount)
    db.save("orders", {**order, "discount": discount, "final": final})
    return final

```

```

# — Техніка 2: Replace Magic Number with Symbolic Constant —————
# BEFORE: магічні числа
def get_discount(user_type, total):
    if user_type == 2:          # що означає 2?
        return total * 0.15
    if total > 1000:
        return total * 0.05

# AFTER: константи або Enum
from enum import IntEnum

```

```

class UserType(IntEnum):
    REGULAR = 1
    VIP      = 2
    PARTNER  = 3

VIP_DISCOUNT_RATE      = 0.15
BULK_DISCOUNT_RATE     = 0.05
BULK_ORDER_THRESHOLD    = 1000.0

def get_discount(user_type: UserType, total: float) -> float:
    if user_type == UserType.VIP:
        return total * VIP_DISCOUNT_RATE
    if total > BULK_ORDER_THRESHOLD:
        return total * BULK_DISCOUNT_RATE
    return 0.0

```

```

# — Техніка 3: Replace Conditional with Polymorphism —————
# BEFORE: великий if/elif
def calculate_shipping(order_type, weight):
    if order_type == "standard":
        return weight * 2.5
    elif order_type == "express":
        return weight * 5.0 + 10
    elif order_type == "overnight":
        return weight * 8.0 + 25
    else:
        raise ValueError(f"Невідомий тип: {order_type}")

# AFTER: поліморфізм (Strategy або Factory)
from abc import ABC, abstractmethod

class ShippingCalculator(ABC):
    @abstractmethod
    def calculate(self, weight: float) -> float:
        pass

class StandardShipping(ShippingCalculator):
    def calculate(self, weight: float) -> float:
        return weight * 2.5

class ExpressShipping(ShippingCalculator):
    def calculate(self, weight: float) -> float:
        return weight * 5.0 + 10.0

class OvernightShipping(ShippingCalculator):
    def calculate(self, weight: float) -> float:
        return weight * 8.0 + 25.0

```



```

SHIPPING_CALCULATORS = {
    "standard": StandardShipping(),
    "express": ExpressShipping(),
    "overnight": OvernightShipping(),
}

def calculate_shipping(order_type: str, weight: float) -> float:
    calc = SHIPPING_CALCULATORS.get(order_type)
    if not calc:
        raise ValueError(f"Невідомий тип доставки: {order_type}")
    return calc.calculate(weight)

```

## 1.5. Інструменти статичного аналізу

| Інструмент                 | Призначення та як використовувати                                                           |
|----------------------------|---------------------------------------------------------------------------------------------|
| <b>SonarLint (VS Code)</b> | Розширення: знаходить bugs, code smells, security issues прямо в редакторі в реальному часі |
| <b>Pylint / Flake8</b>     | CLI: перевіряє Python-код на відповідність PEP 8, знаходить невикористані змінні, імпорти   |
| <b>ESLint</b>              | Статичний аналіз JavaScript/TypeScript. Налаштовується через .eslintrc                      |
| <b>mypy</b>                | Перевіряє type hints у Python: mypy src/                                                    |
| <b>Bandit</b>              | Аналіз безпеки Python-коду: bandit -r src/                                                  |
| <b>SonarCloud</b>          | Безкоштовний для open source репозиторіїв. Повний аналіз після кожного push                 |

## 2. Практичне завдання ПР-11

### ПРАКТИЧНЕ ЗАВДАННЯ 11.1 Code Review наданого коду та рефакторинг свого

#### КРОК 1 Нова гілка

```
git switch -c feature/code-review-refactoring
```

#### КРОК 2 Code Review наданого 'брудного' коду

Нижче наведено код для рев'ю. Ваше завдання — знайти мінімум 7 проблем і описати кожну:

```

# — КОД ДЛЯ РЕВ'Ю: task_service_bad.py —————
def do_stuff(x, y, z, a, b, c):
    data = []
    for i in range(len(x)):
        if x[i] != None:
            if x[i]['status'] == 1:
                data.append(x[i])

```

```

        elif x[i]['status'] == 2:
            if x[i]['priority'] > 5:
                data.append(x[i])

result = []
for i in range(len(data)):
    item = data[i]
    if y == True:
        item['price'] = item['price'] * 0.85
    elif y == False:
        if item['price'] > 500:
            item['price'] = item['price'] * 0.95
    result.append(item)
try:
    f = open('log.txt', 'a')
    f.write(str(result))
except:
    pass
return result

class Manager:
    def __init__(self):
        import sqlite3
        self.db = sqlite3.connect('tasks.db')

    def do_task(self, id, name, desc, prio, dead, uid, cat, tags, att,
noti):
        c = self.db.cursor()
        c.execute("SELECT * FROM tasks WHERE id=" + str(id))
        t = c.fetchone()
        if t != None:
            c.execute("UPDATE tasks SET name='" + name +
                "' WHERE id=" + str(id))
            self.db.commit()
        c.execute("INSERT INTO logs VALUES (?, ?, ?)",
            (id, name, "updated"))
        self.db.commit()
        return True

```

Заповніть таблицю (мінімум 7 рядків):

| Рядок(и)  | Проблема (назва) | Пояснення та рішення |
|-----------|------------------|----------------------|
| ~~рядок~~ | ~~(знайдіть)~~   | ~~(ваше пояснення)~~ |
| ~~рядок~~ | ~~(знайдіть)~~   | ~~(ваше пояснення)~~ |
| ~~рядок~~ | ~~(знайдіть)~~   | ~~(ваше пояснення)~~ |
| ~~рядок~~ | ~~(знайдіть)~~   | ~~(ваше пояснення)~~ |
| ~~рядок~~ | ~~(знайдіть)~~   | ~~(ваше пояснення)~~ |

|           |                |                      |
|-----------|----------------|----------------------|
| ~~рядок~~ | ~~(знайдіть)~~ | ~~(ваше пояснення)~~ |
| ~~рядок~~ | ~~(знайдіть)~~ | ~~(ваше пояснення)~~ |

### КРОК 3 Переписати do\_stuff() після рев'ю

Перепишіть функцію do\_stuff() у файл src/refactored/task\_filter.py. Вимоги:

- Зрозуміла назва функції та параметрів
- Заміна магічних чисел (1, 2, 5, 0.85, 0.95, 500) на константи або Enum
- Розбиття на підфункції (Extract Function)
- Обробка помилок замість 'except: pass'
- Виправлення SQL Injection у класі Manager

### КРОК 4 Рефакторинг вашого коду з ПР-8 або ПР-9

Відкрийте будь-який файл, написаний у попередніх практичних роботах. Знайдіть мінімум 3 Code Smells. Проведіть рефакторинг:

- Перед: скопіюйте оригінал у src/before\_refactoring/
- Після: покращений код у src/ (основне місце)
- У коміті: опишіть які саме зміни зроблено

### КРОК 5 Встановити SonarLint та перевірити

У VS Code: Extensions → пошук 'SonarLint' → встановити. Відкрийте будь-який .py або .js файл вашого проєкту. SonarLint підсвітить проблеми прямо в редакторі. Виправте мінімум 3 попередження від SonarLint.

### КРОК 6 Взаємне рев'ю (peer review)

Обміняйтесь репозиторіями з сусіднім студентом. Переглянете код один одного і залиште мінімум 3 конструктивних коментарі в GitHub PR (якщо немає партнера — залиште 3 коментарі до власного PR описавши що покращили б).

### КРОК 7 Коміт

```
git add src/refactored/ src/before_refactoring/
git commit -m "refactor: виправити code smells у TaskService після code review"
git push origin feature/code-review-refactoring
```

## РЕЗУЛЬТАТ Очікуваний результат роботи

Артефакти для здачі:

- Заповнена таблиця рев'ю: мінімум 7 знайдених проблем
- `src/refactored/task_filter.py` — переписана функція
- Порівняння до/після для коду з ПР-8 або ПР-9
- Скріншот попереджень SonarLint (до і після виправлення)
- Мінімум 3 коментарі на GitHub PR

## ЗАВДАННЯ ДЛЯ ЗАКРІПЛЕННЯ 11.1 Refactoring Kata

Знайдіть в інтернеті будь-який 'Refactoring Kata' (пошук: 'refactoring kata python github'). Популярні: Gilded Rose Kata, Ugly Trivia Kata, Tennis Refactoring Kata. Виконайте мінімум одне ката, зробивши коміти після кожного кроку рефакторингу. Це — стандартна практика у senior-інтерв'ю.

## ? Запитання для самоперевірки

1. Яка різниця між 'виправленням бага' і 'рефакторингом'? Чому їх не варто робити одночасно в одному PR?
2. SQL Injection у наданому коді: покажіть рядок де він є і поясніть як зловмисник міг би його використати. Як виправити?
3. Чому 'except: pass' — небезпечно? Напишіть сценарій, де це може привести до втрати даних.
4. Що означає 'рефакторинг не змінює поведінку'? Як перевірити, що поведінка не змінилась?

## 1. Теоретична частина

### 1.1. Чому звичайний код 'гальмує'

Синхронний (synchronous) код виконується послідовно: одна операція — потім наступна. Це зручно і зрозуміло. Але уявіть веб-сервер, який обробляє запити так: отримав запит → зробив запит до БД (50 мс) → відправив email (200 мс) → відповів клієнту. За цей час сервер не міг обробляти інші запити. При 100 одночасних користувачах — черга.

#### АНАЛОГІЯ Аналогія з реального життя

Синхронний код — як один офіціант у ресторані. Прийняв замовлення від столика 1 → поніс на кухню → чекав → приніс → тоді пішов до столика 2. При 50 столиках — катастрофа. Асинхронний код — ресторан з системою номерків. Офіціант приймає замовлення → передає на кухню → відразу йде до наступного столика → коли кухня готова, сповіщає → офіціант приносить. Один офіціант обслуговує 50 столиків.

### 1.2. Event Loop — серце асинхронності

Event Loop (цикл подій) — механізм, який дозволяє одному потоку виконання обробляти кілька операцій 'одночасно'. Він не робить речі паралельно — він чергує їх, поки одні операції чекають (I/O, мережа, БД), запускаючи інші.

#### ТЕОРІЯ Ключова теорія

Три типи конкурентності у Python:

- Asyncio (async/await): один потік, Event Loop. Ідеально для I/O-bound задач: веб-запити, БД, файли
- Threading: кілька потоків у одному процесі. Для I/O з блокуючими бібліотеками
- Multiprocessing: кілька процесів. Для CPU-bound задач: розрахунки, ML, обробка зображень

```

# Порівняння: синхронний vs асинхронний код
import time, asyncio

# — СИНХРОННИЙ: Sequential —————
def fetch_user_sync(user_id: int) -> dict:
    time.sleep(0.3) # імітація запиту до БД (300 мс)
    return {"id": user_id, "name": f"User {user_id}"}

def load_all_sync():
    start = time.time()
    users = []
    for i in range(1, 4):
        users.append(fetch_user_sync(i)) # 300мс × 3 = 900мс
    print(f"Синхронно: {time.time()-start:.2f}c") # ~0.90c
    return users

# — АСИНХРОННИЙ: Concurrent I/O —————
async def fetch_user_async(user_id: int) -> dict:
    await asyncio.sleep(0.3) # 'await' — відпускаємо потік
    return {"id": user_id, "name": f"User {user_id}"}

async def load_all_async():
    start = time.time()
    # Запускаємо всі три запити ОДНОЧАСНО (gather)
    users = await asyncio.gather(
        fetch_user_async(1),
        fetch_user_async(2),
        fetch_user_async(3),
    )
    print(f"Асинхронно: {time.time()-start:.2f}c") # ~0.30c
    return users

# Запуск
load_all_sync() # 0.90c
asyncio.run(load_all_async()) # 0.30c — втричі швидше!

```

### 1.3. Синтаксис `async/await`

Ключові слова `async` і `await` — синтаксичний цукор для роботи з корутинами (coroutines) у Python. Правило просте: функція з `await` повинна бути оголошена з `async`.

| Ключове слово / вираз                 | Значення та коли використовувати                                                                      |
|---------------------------------------|-------------------------------------------------------------------------------------------------------|
| <b><code>async def func():</code></b> | Оголосити асинхронну функцію (корутину). Повертає coroutine object при виклику без <code>await</code> |

|                                        |                                                                                      |
|----------------------------------------|--------------------------------------------------------------------------------------|
| <b>await expression</b>                | Призупинити виконання і чекати результату. Тільки всередині <code>async def</code>   |
| <b>asyncio.run(coro)</b>               | Запустити корутину як точку входу ( <code>main</code> ). Один раз на весь застосунок |
| <b>asyncio.gather(*coros)</b>          | Запустити кілька корутин паралельно. Чекати всіх. Повертає список результатів        |
| <b>asyncio.create_task(coro)</b>       | Запустити корутину у фоні (не чекати). Повертає <code>Task</code> -об'єкт            |
| <b>asyncio.sleep(n)</b>                | Асинхронна пауза. Не блокує — Event Loop виконує інші задачі                         |
| <b>asyncio.wait_for(coro, timeout)</b> | Виконати корутину з таймаутом. Кидає <code>asyncio.TimeoutError</code>               |

```
# Практичний приклад: асинхронний API-клієнт
import asyncio
import aiohttp # async HTTP client (pip install aiohttp)

async def fetch_task(session: aiohttp.ClientSession, task_id: int) -> dict:
    """Отримати одну задачу з API"""
    url = f"https://jsonplaceholder.typicode.com/todos/{task_id}"
    async with session.get(url) as response:
        if response.status == 200:
            data = await response.json()
            return data
        return {"error": f"HTTP {response.status}"}

async def fetch_multiple_tasks(task_ids: list) -> list:
    """Отримати кілька задач паралельно"""
    async with aiohttp.ClientSession() as session:
        tasks = [fetch_task(session, tid) for tid in task_ids]
        results = await asyncio.gather(*tasks, return_exceptions=True)
    return results

async def main():
    print("Завантажуємо 5 задач паралельно...")
    import time
    start = time.time()
    tasks = await fetch_multiple_tasks([1, 2, 3, 4, 5])
    elapsed = time.time() - start
    print(f"Отримано {len(tasks)} задач за {elapsed:.2f}s")
    for t in tasks[:3]:
        print(f"  - {t.get('title', t)[:50]}")

asyncio.run(main())
```

## 1.4. Типові помилки в async/await

| Помилка                                                        | Що відбувається та як виправити                                                                                                 |
|----------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Виклик <code>async def</code> без <code>await</code>           | Функція повертає coroutine object, а не результат.<br>Завжди: <code>result = await my_func()</code>                             |
| <code>await</code> поза <code>async def</code>                 | <code>SyntaxError</code> . Функція-контейнер теж має бути <code>async</code>                                                    |
| <code>time.sleep()</code> замість <code>asyncio.sleep()</code> | Блокує весь Event Loop. Замінити на <code>await asyncio.sleep()</code>                                                          |
| <code>requests</code> замість <code>aiohttp</code>             | <code>requests</code> — синхронна бібліотека. Блокує Event Loop.<br>Використовувати <code>aiohttp</code> або <code>httpx</code> |
| Не обробляти виключення в <code>gather</code>                  | Один збій ламає всі. Використовувати <code>return_exceptions=True</code>                                                        |

```
# Типові помилки та їх виправлення

# ❌ ПОМИЛКА 1: виклик async без await
async def get_data(): return 42

result = get_data()      # повертає <coroutine object>, НЕ 42!
print(result)           # <coroutine object get_data at 0x...>

# ✅ ПРАВИЛЬНО
result = await get_data() # всередині async def

# ❌ ПОМИЛКА 2: блокуючий sleep
async def wrong():
    import time
    time.sleep(2)        # ЗАБЛОКУЄ весь Event Loop на 2 секунди!

# ✅ ПРАВИЛЬНО
async def correct():
    await asyncio.sleep(2) # відпускає Event Loop

# ❌ ПОМИЛКА 3: не обробляти помилки в gather
async def might_fail(i):
    if i == 2: raise ValueError("Помилка!")
    return i

# Без return_exceptions — помилка зупиняє всі:
results = await asyncio.gather(
    might_fail(1), might_fail(2), might_fail(3)
) # Падає при i==2, втрачаємо результати 1 і 3

# ✅ З return_exceptions — отримуємо всі результати:
results = await asyncio.gather(
```



```

    might_fail(1), might_fail(2), might_fail(3),
    return_exceptions=True
)
# results = [1, ValueError("Помилка!"), 3]
for r in results:
    if isinstance(r, Exception):
        print(f"Помилка: {r}")
    else:
        print(f"Результат: {r}")

```

## 1.5. Background Tasks — фонові задачі

Фонова задача (background task) — операція, яка виконується паралельно з основним потоком і не блокує відповідь клієнту. Типові випадки: відправка email після реєстрації, генерація PDF-звіту, синхронізація з зовнішнім API, очищення старих даних.

```

# Background Tasks у FastAPI (найпростіший варіант)
from fastapi import FastAPI, BackgroundTasks

app = FastAPI()

def send_welcome_email(email: str, name: str):
    """Ця функція виконується у фоні — може бути довгою"""
    import time
    time.sleep(2) # імітація відправки email
    print(f"Email надіслано на {email}")

@app.post("/users/")
async def create_user(
    data: dict,
    background_tasks: BackgroundTasks # FastAPI ін'єктує автоматично
):
    # 1. Зберегти користувача (швидко)
    user = {"id": 1, "email": data["email"], "name": data["name"]}

    # 2. Додати email до фонові черги (не чекати!)
    background_tasks.add_task(
        send_welcome_email,      # функція
        data["email"],           # аргументи
        data["name"]
    )

    # 3. ВІДРАЗУ повернути відповідь — не чекаючи email
    return {"id": user["id"], "message": "Користувача створено!"}
# Email буде надіслано після того як відповідь вже у клієнта

```

## ІНДУСТРІЯ Як це використовують у роботі

Для серйозних production-систем з тисячами фонових задач використовують Celery (Python) або Bull (Node.js) — повноцінні черги задач. Вони зберігають задачі в Redis або RabbitMQ, можуть повторювати задачі при збоях, показують прогрес і статус. FastAPI BackgroundTasks — для простих випадків без критичних вимог до надійності.

## 2. Практичне завдання ПР-12

### ПРАКТИЧНЕ ЗАВДАННЯ 12.1 Асинхронні ендпоінти та фонові задачі у вашому проєкті

#### КРОК 1 Нова гілка та встановлення залежностей

```
git switch -c feature/async-programming
pip install aiohttp asyncio
# Якщо FastAPI: pip install fastapi uvicorn
# Якщо Flask: pip install flask (sync) або quart (async Flask)
# Зафіксуйте залежності:
pip freeze > requirements.txt
git add requirements.txt
git commit -m "chore: додати aiohttp до залежностей"
```

#### КРОК 2 Базовий async-сервіс — послідовна vs паралельна вибірка

Створіть файл `src/services/async_demo.py`. Реалізуйте дві версії однієї операції і порівняйте час:

```
# src/services/async_demo.py
import asyncio, time

async def simulate_db_query(table: str, id: int, delay: float = 0.2) -> dict:
    """Імітація асинхронного запиту до БД"""
    await asyncio.sleep(delay)
    return {"table": table, "id": id, "data": f"Дані з {table} [{id}]}"}

async def load_task_with_user_SLOW(task_id: int) -> dict:
    """ПОВІЛЬНО: послідовні запити"""
    task = await simulate_db_query("tasks", task_id, 0.3)
    user = await simulate_db_query("users", task["id"], 0.2)
    comments = await simulate_db_query("comments", task["id"], 0.15)
    return {"task": task, "user": user, "comments": comments}
    # Загальний час: 0.3 + 0.2 + 0.15 = 0.65 сек

async def load_task_with_user_FAST(task_id: int) -> dict:
```

```

"""ШВИДКО: паралельні запити через gather"""
task, user, comments = await asyncio.gather(
    simulate_db_query("tasks", task_id, 0.3),
    simulate_db_query("users", task_id, 0.2),
    simulate_db_query("comments", task_id, 0.15),
)
return {"task": task, "user": user, "comments": comments}
# Загальний час: max(0.3, 0.2, 0.15) = 0.30 сек

async def benchmark():
    print("Порівняння: Sequential vs Parallel")
    t0 = time.perf_counter()
    slow = await load_task_with_user_SLOW(1)
    t1 = time.perf_counter()
    fast = await load_task_with_user_FAST(1)
    t2 = time.perf_counter()

    print(f"Послідовно: {t1-t0:.3f} сек")
    print(f"Паралельно: {t2-t1:.3f} сек")
    print(f"Прискорення: {(t1-t0)/(t2-t1):.1f}x")

if __name__ == "__main__":
    asyncio.run(benchmark())

```

### КРОК 3 Async ендпоінт у вашому проєкті

Виберіть один сервіс з вашого проєкту і зробіть його async. Мінімально:

- Один `async def` ендпоінт або функція, яка виконує 2+ I/O операції
- Використати `asyncio.gather()` для паралельної вибірки
- Обробка виключень через `return_exceptions=True` або `try/except`

Якщо ваш проєкт на FastAPI — додайте `async def` до роутера. Якщо на Flask — окрема async-функція з `asyncio.run()`.

### КРОК 4 Реалізувати фонову задачу

Визначте одну операцію у вашому проєкті, яка підходить для фонового виконання (відправка повідомлення, логування, генерація файлу). Реалізуйте:

- Функцію (може бути синхронною), що виконує цю операцію
- Запуск через `BackgroundTasks` (FastAPI) або `asyncio.create_task()`
- Логування початку і кінця фонові задачі

```

# Приклад для FastAPI
from fastapi import BackgroundTasks

def log_task_completion(task_id: int, user_id: int):
    import logging
    logger = logging.getLogger(__name__)
    logger.info(f"[BG] Фонова задача: task #{task_id} завершена user #{user_id}")
    # Тут може бути відправка email, запис статистики тощо

@router.patch("/tasks/{task_id}/complete")
async def complete_task(
    task_id: int,
    user_id: int,
    bg: BackgroundTasks
):
    # Основна логіка (швидка)
    result = await task_service.mark_done(task_id, user_id)
    # Фонова операція (не чекаємо)
    bg.add_task(log_task_completion, task_id, user_id)
    return result # відповідь — миттєво

```

## КРОК 5 Тест асинх функцій

Async функції теж тестуються — через `pytest-asyncio` (`pip install pytest-asyncio`):

```

# tests/test_async_service.py
import pytest, asyncio
from src.services.async_demo import (
    load_task_with_user_SLOW,
    load_task_with_user_FAST,
    benchmark
)

@pytest.mark.asyncio
async def test_parallel_faster_than_sequential():
    """Перевіряємо що паралельна версія швидша"""
    import time
    t0 = time.perf_counter()
    await load_task_with_user_SLOW(1)
    slow_time = time.perf_counter() - t0

    t0 = time.perf_counter()
    await load_task_with_user_FAST(1)
    fast_time = time.perf_counter() - t0

    # Паралельна версія повинна бути хоча б у 1.5 рази швидша
    assert fast_time < slow_time / 1.5, (
        f"Очікували прискорення, але: slow={slow_time:.3f}, "
        f"fast={fast_time:.3f}"
    )

```

```
@pytest.mark.asyncio
async def test_fast_returns_all_fields():
    result = await load_task_with_user_FAST(1)
    assert "task" in result
    assert "user" in result
    assert "comments" in result
```

#### КРОК 6 Виміряйте та задокументуйте прискорення

Запустіть benchmark:

```
python src/services/async_demo.py
```

Зробіть скріншот виводу. Додайте у docs/ файл async\_benchmark.md зі скріншотом і поясненням отриманого прискорення.

#### КРОК 7 Коміт та PR

```
git add src/ tests/ docs/
git commit -m "feat: додати async ендпоінти та фонові задачі"
git push origin feature/async-programming
```

### РЕЗУЛЬТАТ Очікуваний результат роботи

Артефакти для задачі:

- src/services/async\_demo.py — benchmark sequential vs parallel
- Мінімум один async ендпоінт або функція з asyncio.gather()
- Реалізована фонові задача
- 2+ async тести з pytest-asyncio
- docs/async\_benchmark.md — скріншот з результатами

#### ЗАВДАННЯ ДЛЯ ЗАКРІПЛЕННЯ 12.1 Таймаут та retry для async

Реалізуйте async функцію з таймаутом і повтором при помилці:

```
async def fetch_with_retry(url: str, max_attempts: int = 3) -> dict:
    """
    Завдання: реалізуйте функцію, яка:
    1. Намагається виконати HTTP GET запит
    2. При помилці або таймауті (3 сек) — повторює спробу
    3. Після max_attempts невдалих спроб — кидає Exception
```

```
4. Між спробами – затримка: 1с, 2с, 4с (exponential backoff)
"""
for attempt in range(1, max_attempts + 1):
    try:
        # Ваш код тут: asyncio.wait_for(fetch(url), timeout=3)
        ...
    except (asyncio.TimeoutError, aiohttp.ClientError) as e:
        if attempt == max_attempts:
            raise
        delay = 2 ** (attempt - 1)
        await asyncio.sleep(delay)
```

## ЗАВДАННЯ ДЛЯ ЗАКРІПЛЕННЯ 12.2 Producers та Consumers через asyncio.Queue

Реалізуйте простий producer-consumer на asyncio.Queue: Producer генерує 10 задач із затримкою 0.1 сек. 3 Consumer-и паралельно обробляють задачі (кожна обробка — 0.2 сек). Порівняйте загальний час з послідовною обробкою.

Підказка: asyncio.Queue, asyncio.create\_task(), await queue.get(), queue.task\_done().

## ? Запитання для самоперевірки

1. Яка різниця між async/await і threading? Коли краще кожен підхід? Наведіть конкретний приклад задачі для кожного.
2. Чому asyncio.gather() може бути небезпечним без return\_exceptions=True? Що трапиться якщо одна з 5 корутин кине виключення?
3. У вашому проєкті: які операції є кандидатами для асинхронного виконання? Чому саме вони?
4. Поясніть аналогією (не технічно) різницю між asyncio.gather() і asyncio.create\_task(). Коли потрібен кожен?

## Підсумок модуля 3

Модуль 3 став найнасиченішим кодом. Ви пройшли шлях від патернів, що організують створення об'єктів, до асинхронного коду, що виконує їх одночасно.

| Компетентність                                    | Як перевірити себе                                               |
|---------------------------------------------------|------------------------------------------------------------------|
| Застосувати Singleton, Factory, Builder           | Реалізуйте всі три у новому контексті за 30 хвилин без підказок  |
| Застосувати Observer, Strategy, Decorator, Facade | Поясніть який патерн вирішує конкретну проблему з вашого проекту |
| Побудувати ієрархію виключень                     | Намалюйте ієрархію вашого проекту і поясніть кожен клас          |
| Налаштувати структуроване логування               | Запустіть проєкт і покажіть JSON-логи у терміналі                |
| Провести code review                              | Знайдіть 5 code smells у наданому коді за 15 хвилин              |
| Зробити рефакторинг                               | Спростіть 20-рядкову функцію без зміни поведінки                 |
| Написати async функцію з gather                   | Отримайте дані з 3 джерел паралельно                             |
| Реалізувати фонову задачу                         | Додайте до ендпоінту фонову операцію що не блокує відповідь      |

### ТЕОРІЯ Ключова теорія

Ключові поняття модуля 3: Design Pattern (патерн проектування), GoF (Gang of Four), Creational Patterns (Singleton / Factory Method / Builder), Behavioral Patterns (Observer / Strategy), Structural Patterns (Decorator / Facade), Exception Hierarchy (ієрархія виключень), Fail Fast / Fail Loudly, Input Validation (валідація вхідних даних), Structured Logging (структуроване логування), Log Levels (DEBUG/INFO/WARNING/ERROR/CRITICAL), Code Review, Code Smells, Refactoring, Extract Function, Magic Number, Synchronous vs Asynchronous, Event Loop, async/await, asyncio.gather(), Background Tasks, Coroutine.

Модуль 4 завершить курс: тестування (Unit та Integration), CI/CD з GitHub Actions, Docker та фінальний захист вашого наскрізного проєкту.